Numerical Methods

Lecture 5-7 Linear Algebra (Matrix)

1. 考试重点

和之前的内容不一样，矩阵部分不光要求能写代码还要求能用公式手写出答案。 题目对于矩阵和线下方程求解有深刻的要求，务必理解下面的全部内容

1.1 矩阵的基本运算：

包括矩阵的乘法、矩阵的加法、矩阵的转置等。

1.2 矩阵的性质：

- 矩阵的行列式：用于判断矩阵是否可逆，以及线性方程组是否有唯一解。
- 矩阵的逆：通过行变换将矩阵化为单位矩阵的同时，单位矩阵会变为原矩阵的逆矩阵。
- 线性方程组的解的性质：包括线性方程组有唯一解、无解和有无穷多解三种情况。

1.3 线性方程组的解法：

- 高斯消元法：通过行变换将线性方程组的增广矩阵化为上三角形式，然后利用回代求解。
- LU分解法：将矩阵分解为一个下三角矩阵和一个上三角矩阵的乘积，然后利用LU分解求解线性方程组。
- 迭代法：如高斯-赛德尔迭代法，通过迭代的方式逐步逼近线性方程组的解。

2. Basic Matrix Operations and Properties（矩阵的基本运算和性质）

2.1 Basic Matrix Operations（矩阵的基本运算）

2.1.1 Matrix Addition and Subtraction（矩阵的加法和减法）

Matrix addition and subtraction are performed element-wise. Given two matrices A and B of the same dimensions, the sum $C = A + B$ and the difference $D = A - B$ are defined as:

$$C_{ij} = A_{ij} + B_{ij}, \quad D_{ij} = A_{ij} - B_{ij}$$

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}, \quad A - B = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

2.1.2 Matrix Multiplication（矩阵的乘法）

Matrix multiplication is defined for matrices A of size $m \times n$ and B of size $n \times p$. The product $C = AB$ is a matrix of size $m \times p$ with elements:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

2.1.3 Matrix Transpose（矩阵的转置）

The transpose of a matrix A, denoted A^T, is obtained by swapping the rows and columns of A. If A is an $m \times n$ matrix, then A^T is an $n \times m$ matrix with elements:

$$(A^T)_{ij} = A_{ji}$$

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

2.2 Matrix Properties（矩阵的性质）

2.2.1 Determinant（行列式）

The determinant of a square matrix A, denoted $\det(A)$ or $|A|$, is a scalar value that can be computed from the elements of A. It is used to determine if the matrix is invertible（可逆）. If $\det(A) \neq 0$, the matrix is invertible; otherwise, it is not.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\det(A) = 1 \cdot 4 - 2 \cdot 3 = -2$$

Since $\det(A) \neq 0$, A is invertible.

2.2.2 Inverse（逆矩阵）

The inverse of a square matrix A, denoted A^{-1}, is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix（单位矩阵）. The inverse exists if and only if $\det(A) \neq 0$.

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} = \frac{1}{-2} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix}$$

2.2.3 Solutions of Linear Systems（线性方程组的解）

A linear system $A\boldsymbol{x} = \boldsymbol{b}$ can have:

- **Unique Solution**（唯一解）: If $\det(A) \neq 0$, the system has a unique solution.
- **No Solution**（无解）: If the system is inconsistent.
- **Infinitely Many Solutions**（无穷多解）: If the system is dependent.

3. Calculation（计算方法）

3.1 Gaussian Elimination（高斯消元法）

3.1.1 Overview

Gaussian Elimination is a method for solving systems of linear equations by transforming the augmented matrix（增广矩阵）into an upper-triangular form（上三角形式）. This process involves a series of row operations（行操作）to simplify the matrix, making it easier to solve for the unknowns using back substitution（回代）.

3.1.2 Step-by-Step Process

Consider the following system of linear equations:

$$x + 2y + 4z = -20 \tag{1}$$
$$2x + 5y + 6z = -36 \tag{2}$$
$$-x - 5y + 3z = 26 \tag{3}$$

3.1.2.1 Write the Augmented Matrix（写出增广矩阵）

First, we write the augmented matrix for the system:

$$\left[\begin{array}{ccc|c} 1 & 2 & 4 & -20 \\ 2 & 5 & 6 & -36 \\ -1 & -5 & 3 & 26 \end{array}\right]$$

3.1.2.2 Eliminate the First Column Below the Pivot（消去第一列的主元下方元素）

The goal is to make the elements below the first pivot（主元）equal to zero. We use row operations to achieve this:

- Subtract 2 times the first row from the second row:

$$R2 \leftarrow R2 - 2R1$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 4 & -20 \\ 0 & 1 & -2 & 4 \\ -1 & -5 & 3 & 26 \end{array}\right]$$

- Add the first row to the third row:

$$R3 \leftarrow R3 + R1$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 4 & -20 \\ 0 & 1 & -2 & 4 \\ 0 & -3 & 7 & 6 \end{array}\right]$$

Explanation: The purpose of these operations is to create zeros below the first pivot (1 in the first row). This simplifies the matrix and brings it closer to upper-triangular form.

3.1.2.3 Eliminate the Second Column Below the Pivot（消去第二列的主元下方元素）

Next, we eliminate the elements below the second pivot:

- Add 3 times the second row to the third row:

$$R3 \leftarrow R3 + 3R2$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 4 & -20 \\ 0 & 1 & -2 & 4 \\ 0 & 0 & 1 & 18 \end{array}\right]$$

Explanation: By adding a multiple of the second row to the third row, we create a zero below the second pivot (1 in the second row). This step further simplifies the matrix and brings it closer to upper-triangular form.

3.1.2.4 Back Substitution（回代求解）

Now that the matrix is in upper-triangular form, we can solve for the unknowns using back substitution:

- Solve for z:

$$z = 18$$

- Solve for y:

$$y - 2z = 4 \implies y - 2(18) = 4 \implies y = 40$$

- Solve for x:

$$x + 2y + 4z = -20 \implies x + 2(40) + 4(18) = -20 \implies x = -172$$

Explanation: Back substitution starts from the bottom row and works its way up. Since the matrix is upper-triangular, each equation has only one unknown variable that has not been solved yet, making it easy to find the values step by step.

3.1.2.5 Summary

The solution to the system is:

$$\boxed{x = -172, y = 40, z = 18}$$

3.2 LU Decomposition（LU分解法）

3.2.1 Overview

LU Decomposition is a technique that decomposes a matrix A into a lower-triangular matrix L（下三角矩阵）and an upper-triangular matrix U（上三角矩阵）such that $A = LU$. This decomposition allows us to solve the system $A\boldsymbol{x} = \boldsymbol{b}$ more efficiently by solving two simpler triangular systems.

3.2.2 Step-by-Step Process

Consider the matrix A:

$$A = \begin{bmatrix} 2 & 3 & 1 & 5 \\ 8 & 16 & 7 & 22 \\ 2 & 15 & 13 & 19 \\ -4 & 2 & 10 & 11 \end{bmatrix}$$

3.2.2.1 Perform Row Operations（执行行操作）

We will perform row operations to transform A into an upper-triangular matrix U, while recording the operations to construct the lower-triangular matrix L.

Step 1: Eliminate the first column below the pivot

- Subtract 4 times the first row from the second row:

$$R2 \leftarrow R2 - 4R1$$

- Add the first row to the third row:

$$R3 \leftarrow R3 - R1$$

- Add 2 times the first row to the fourth row:

$$R4 \leftarrow R4 + 2R1$$

Step 2: Eliminate the second column below the pivot

- Subtract 3 times the second row from the third row:

$$R3 \leftarrow R3 - 3R2$$

- Subtract 2 times the second row from the fourth row:

$$R4 \leftarrow R4 - 2R2$$

Step 3: Eliminate the third column below the pivot

- Subtract 2 times the third row from the fourth row:

$$R4 \leftarrow R4 - 2R3$$

3.2.2.2 Construct L and U（构造 L 和 U）

After performing the row operations, we obtain:

$$U = \begin{bmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 3 & 2 \\ 0 & 0 & 3 & 8 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The lower-triangular matrix L is constructed by recording the multipliers used in the row operations:

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 4 & 1 & 0 & 0 \\ 1 & 3 & 1 & 0 \\ -2 & 2 & 2 & 1 \end{bmatrix}$$

Explanation: The matrix L records the steps we took to transform A into U. Each entry below the diagonal in L corresponds to a multiplier used in the row operations.

3.2.2.3 Solve the System Using L and U（利用 L 和 U 求解线性方程组）

Given a right-hand side vector $\boldsymbol{b}$, we solve the system $A\boldsymbol{x} = \boldsymbol{b}$ by solving two triangular systems:

1. Solve $L\boldsymbol{c} = \boldsymbol{b}$ for $\boldsymbol{c}$ using forward substitution（前代）.
2. Solve $U\boldsymbol{x} = \boldsymbol{c}$ for $\boldsymbol{x}$ using back substitution（回代）.

Example: Let $\boldsymbol{b} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$.

Step 1: Solve $L\boldsymbol{c} = \boldsymbol{b}$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 4 & 1 & 0 & 0 \\ 1 & 3 & 1 & 0 \\ -2 & 2 & 2 & 1 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

Step 2: Solve $U\boldsymbol{x} = \boldsymbol{c}$

$$\begin{bmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 3 & 2 \\ 0 & 0 & 3 & 8 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix}$$

3.3 Gauss-Seidel Iteration（高斯-赛德尔迭代法）

3.3.1 Overview

Gauss-Seidel Iteration is an iterative method for solving systems of linear equations. It updates the solution iteratively by using the latest available values for each variable.

3.3.2 Step-by-Step Process

Consider the following system of linear equations:

$$x + 2y + 4z = -20 \tag{4}$$
$$2x + 5y + 6z = -36 \tag{5}$$
$$-x - 5y + 3z = 26 \tag{6}$$

3.3.2.1 Rewrite the Equations（重写方程）

Rewrite each equation to solve for one variable:

$$x = -20 - 2y - 4z \tag{7}$$
$$y = \frac{-36 - 2x - 6z}{5} \tag{8}$$
$$z = \frac{26 + x + 5y}{3} \tag{9}$$

3.3.2.2 Initialize the Solution（初始化解）

Start with an initial guess for the solution, such as $\boldsymbol{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.

3.3.2.3 Iterate（迭代）

Update the solution iteratively:

$$x^{(k+1)} = -20 - 2y^{(k)} - 4z^{(k)} \tag{10}$$
$$y^{(k+1)} = \frac{-36 - 2x^{(k+1)} - 6z^{(k)}}{5} \tag{11}$$
$$z^{(k+1)} = \frac{26 + x^{(k+1)} + 5y^{(k+1)}}{3} \tag{12}$$

Example: Perform a few iterations starting from $\boldsymbol{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.

Iteration 1:

$$x^{(1)} = -20 - 2(0) - 4(0) = -20 \tag{13}$$
$$y^{(1)} = \frac{-36 - 2(-20) - 6(0)}{5} = \frac{-36 + 40}{5} = 0.8 \tag{14}$$
$$z^{(1)} = \frac{26 + (-20) + 5(0.8)}{3} = \frac{26 - 20 + 4}{3} = \frac{10}{3} \approx 3.33 \tag{15}$$

Iteration 2:

$$x^{(2)} = -20 - 2(0.8) - 4(3.33) = -20 - 1.6 - 13.32 = -34.92$$
$$y^{(2)} = \frac{-36 - 2(-34.92) - 6(3.33)}{5} = \frac{-36 + 69.84 - 19.98}{5} = \frac{13.86}{5} = 2.$$
$$z^{(2)} = \frac{26 + (-34.92) + 5(2.772)}{3} = \frac{26 - 34.92 + 13.86}{3} = \frac{4.94}{3} \approx 1.65$$

Explanation: Each iteration updates the solution using the latest values. The process continues until the solution converges to a stable value.

3.3.2.4 Convergence（收敛性）

Gauss-Seidel Iteration converges if the matrix A is strictly diagonally dominant（严格对角占优）or symmetric and positive definite（对称正定）. In practice, the iteration is stopped when the change in the solution is below a certain tolerance.

3.4 Additional Examples with Jupyter-Compatible Format

Example 1: Solving a System Using Gaussian Elimination

Consider the system:

$$x + 2y - 2z = 7 \tag{19}$$
$$-x + 0y + 8z = 7 \tag{20}$$
$$2x + 6y + 2z = 30 \tag{21}$$

The augmented matrix is:

$$\left[\begin{array}{ccc|c} 1 & 2 & -2 & 7 \\ -1 & 0 & 8 & 7 \\ 2 & 6 & 2 & 30 \end{array}\right]$$

Step 1: Eliminate first column below pivot

$$R_2 \leftarrow R_2 + R_1$$
$$R_3 \leftarrow R_3 - 2R_1$$
$$\left[\begin{array}{ccc|c} 1 & 2 & -2 & 7 \\ 0 & 2 & 6 & 14 \\ 0 & 2 & 6 & 16 \end{array}\right]$$

Step 2: Eliminate second column below pivot

$$R_3 \leftarrow R_3 - R_2$$
$$\left[\begin{array}{ccc|c} 1 & 2 & -2 & 7 \\ 0 & 2 & 6 & 14 \\ 0 & 0 & 0 & 2 \end{array}\right]$$

Since the last row represents $0 = 2$, this system has **no solution**.

Example 2: Matrix Operations

Given matrices:

$$A = \begin{bmatrix} 1 & 3 & 2 \\ 4 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 2 \end{bmatrix}$$

Matrix Multiplication AB:

$$AB = \begin{bmatrix} 1 & 3 & 2 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 1 & 2 \end{bmatrix}$$

$$AB = \begin{bmatrix} 1(2) + 3(0) + 2(1) & 1(1) + 3(3) + 2(2) \\ 4(2) + 0(0) + 1(1) & 4(1) + 0(3) + 1(2) \end{bmatrix}$$

$$AB = \begin{bmatrix} 4 & 14 \\ 9 & 6 \end{bmatrix}$$

4. 代码实现

4.1 Gaussian Elimination

In [1]:

```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.linalg as sl
import scipy.optimize as sop

# 此处记得更换数据，A 是方程左边各变量的系数矩阵，b 是右边的常数向量
# Gaussian Elimination
# A is the coefficient matrix on the left-hand side, and b is the constan
# 2x2 example
A = np.array([[2., 3.],
              [1., -4.]])
b = np.array([7., 3.])

"""把矩阵 A 通过行变换化成上三角矩阵，同时对 b 做相同操作"""
# Convert matrix A into upper triangular form using row operations, and a
def upper_triangle(A, b):

    n = np.size(b)
    rows, cols = np.shape(A)

    # check if A is square
    assert(rows == cols)

    # check if the number of rows in A matches the size of b
    assert(rows == n)

    # Loop over each pivot row (except the last one)
    for k in range(n - 1):

        # Eliminate entries below the pivot
```

```
        for i in range(k + 1, n):

            # Elimination factor
            s = A[i, k] / A[k, k]

            # Update the current row of A
            for j in range(k, n):
                A[i, j] = A[i, j] - s * A[k, j]

            # Update b with the same row operation
            b[i] = b[i] - s * b[k]

"""对上三角矩阵形式的 Ax = b 做回代，返回解向量 x"""
# Perform back substitution for an upper triangular system Ax = b and ret
def back_substitution(A, b):

    n = np.size(b)

    # Check whether A is square and dimensions are consistent
    rows, cols = np.shape(A)
    assert(rows == cols)
    assert(rows == n)

    # Check whether A is upper triangular
    assert(np.allclose(A, np.triu(A)))

    # Initialize the solution vector
    x = np.zeros(n)

    # Start back substitution from the last row
    for k in range(n - 1, -1, -1):
        s = 0.0

        # Compute the sum of known terms
        for j in range(k + 1, n):
            s = s + A[k, j] * x[j]

        # Solve for the current unknown
        x[k] = (b[k] - s) / A[k, k]

    return x

# 这里的 A 已经是上三角矩阵
# This A is already in upper triangular form
A = np.array([[ 2.,  3., -4.],
              [ 0., -1., 14.],
              [ 0.,  0., 30.]])
b = np.array([5., -12., -15.])

# 用我们自己写的代码求解
# Solve using our own code
x = back_substitution(A, b)
print('Our solution: ', x)

# 重新初始化 A 和 b
# Reinitialize A and b
A = np.array([[2., 3., -4.],
              [6., 8.,  2.],
              [4., 8., -6.]])
b = np.array([5., 3., 19.])
```

```
# 用 SciPy 的逆矩阵方法求解并对比
# Solve using SciPy's inverse-based method and compare
print('SciPy solution: ', sl.inv(A) @ b)

# 检查两种结果是否一致
# Check whether the two solutions agree
print('Success: ', np.allclose(x, sl.inv(A) @ b))
```

```
---------------------------------------------------------------------------
ImportError                               Traceback (most recent call last)
File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/__init__.py:23
     22 try:
---> 23     from . import multiarray
     24 except ImportError as exc:

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/multiarray.py:10
      9 import functools
---> 10 from . import overrides
     11 from . import _multiarray_umath

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/overrides.py:8
      7 from ..._utils._inspect import getargspec
----> 8 from numpy._core._multiarray_umath import (
      9     add_docstring,  _get_implementing_args, _ArrayFunctionDispatcher)
     12 ARRAY_FUNCTIONS = set()

ImportError: dlopen(/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/_multiarray_umath.cpython-312-darwin.so, 0x0002): Library not loaded: @rpath/libgfortran.5.dylib
  Referenced from: <0B9C315B-A1DD-3527-88DB-4B90531D343F> /Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/libopenblas.0.dylib
  Reason: tried: '/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/../../../../libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/_core/../../../../libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/bin/../lib/libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/bin/../lib/libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/usr/local/lib/libgfortran.5.dylib' (no such file), '/usr/lib/libgfortran.5.dylib' (no such file, not in dyld cache)

During handling of the above exception, another exception occurred:

ImportError                               Traceback (most recent call last)
File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/__init__.py:114
    113 try:
--> 114     from numpy.__config__ import show as show_config
    115 except ImportError as e:

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/__config__.py:4
      3 from enum import Enum
----> 4 from numpy._core._multiarray_umath import (
      5     __cpu_features__,
      6     __cpu_baseline__,
      7     __cpu_dispatch__,
```

```
      8 )
     10 __all__ = ["show"]

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-package
s/numpy/_core/__init__.py:49
     26     msg = """
     27
     28 IMPORTANT: PLEASE READ THIS FOR ADVICE ON HOW TO SOLVE THIS ISSUE!
   (...)
     47 """ % (sys.version_info[0], sys.version_info[1], sys.executable,
     48         __version__, exc)
---> 49     raise ImportError(msg)
     50 finally:

ImportError:

IMPORTANT: PLEASE READ THIS FOR ADVICE ON HOW TO SOLVE THIS ISSUE!

Importing the numpy C-extensions failed. This error can happen for
many reasons, often due to issues with your setup or how NumPy was
installed.

We have compiled some common reasons and troubleshooting tips at:

    https://numpy.org/devdocs/user/troubleshooting-importerror.html

Please note and check the following:

  * The Python version is: Python3.12 from "/Users/lzl/Library/jupyterlab-
desktop/jlab_server/bin/python"
  * The NumPy version is: "2.1.0"

and make sure that they are the versions you expect.
Please carefully study the documentation linked above for further help.

Original error was: dlopen(/Users/lzl/Library/jupyterlab-desktop/jlab_serv
er/lib/python3.12/site-packages/numpy/_core/_multiarray_umath.cpython-312-
darwin.so, 0x0002): Library not loaded: @rpath/libgfortran.5.dylib
  Referenced from: <0B9C315B-A1DD-3527-88DB-4B90531D343F> /Users/lzl/Libra
ry/jupyterlab-desktop/jlab_server/lib/libopenblas.0.dylib
  Reason: tried: '/Users/lzl/Library/jupyterlab-desktop/jlab_server/lib/li
bgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Librar
y/jupyterlab-desktop/jlab_server/lib/libgfortran.5.dylib' (duplicate LC_RP
ATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/li
b/python3.12/site-packages/numpy/_core/../../../../libgfortran.5.dylib' (d
uplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/
jlab_server/lib/python3.12/site-packages/numpy/_core/../../../../libgfortr
an.5.dylib' (duplicate LC_RPATH '@loader_path'), '/Users/lzl/Library/jupyt
erlab-desktop/jlab_server/bin/../lib/libgfortran.5.dylib' (duplicate LC_RP
ATH '@loader_path'), '/Users/lzl/Library/jupyterlab-desktop/jlab_server/bi
n/../lib/libgfortran.5.dylib' (duplicate LC_RPATH '@loader_path'), '/usr/l
ocal/lib/libgfortran.5.dylib' (no such file), '/usr/lib/libgfortran.5.dyli
b' (no such file, not in dyld cache)


The above exception was the direct cause of the following exception:

ImportError                               Traceback (most recent call las
t)
Cell In[1], line 1
```

```
----> 1 import matplotlib.pyplot as plt
      2 import numpy as np
      3 import scipy.linalg as sl

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/matplotlib/__init__.py:159
    155 from packaging.version import parse as parse_version
    157 # cbook must import matplotlib only within function
    158 # definitions, so it is safe to import from it here.
--> 159 from . import _api, _version, cbook, _docstring, rcsetup
    160 from matplotlib.cbook import sanitize_sequence
    161 from matplotlib._api import MatplotlibDeprecationWarning

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/matplotlib/cbook.py:24
     21 import types
     22 import weakref
---> 24 import numpy as np
     26 try:
     27     from numpy.exceptions import VisibleDeprecationWarning  # numpy >= 1.25

File ~/Library/jupyterlab-desktop/jlab_server/lib/python3.12/site-packages/numpy/__init__.py:119
    115 except ImportError as e:
    116     msg = """Error importing numpy: you should not try to import numpy from
    117     its source directory; please exit the numpy source tree, and relaunch
    118     your python interpreter from there."""
--> 119     raise ImportError(msg) from e
    121 from . import _core
    122 from ._core import (
    123     False_, ScalarType, True_, _get_promotion_state, _no_nep50_warning,
    124     _set_promotion_state, abs, absolute, acos, acosh, add, all, allclose,
   (...)
    169     vstack, where, zeros, zeros_like
    170 )

ImportError: Error importing numpy: you should not try to import numpy from
        its source directory; please exit the numpy source tree, and relaunch
        your python interpreter from there.
```

4.2 LU Compression

In [ ]:

```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.linalg as sl
import scipy.optimize as sop

# 此处记得更换数据，A 是方程左边各变量的系数矩阵，b 是右边的常数向量
# LU Compression / LU Decomposition
# A is the coefficient matrix on the left-hand side and b is the constant
A = np.array([[1.,  2.,  4.],
              [2.,  5.,  6.],
```

```
                [-1., -5.,  3.]])
b = np.array([-20., -36., 26.])

"""这个函数把原矩阵 A 分解成下三角矩阵 L 和上三角矩阵 U,
也就是实现 A = LU，本方法本质上是高斯消元，并把消元过程中的倍数存进 L"""
# This function decomposes the original matrix A into a lower triangular
def LU_decomposition(A):

    # Make a copy of A so that the original matrix is not modified
    A = A.copy()

    # Get the shape of the matrix
    m, n = A.shape

    # Check whether A is square
    assert(m == n)

    # Initialize L as a zero matrix
    # If memory efficiency were important, we could reuse the zeroed entr
    L = np.zeros((n, n))

    # Loop over each pivot row (the final row does not need to be a pivot
    for k in range(n - 1):

        # Eliminate the entries below the pivot
        for i in range(k + 1, n):

            # Compute the elimination multiplier
            # This multiplier will later be stored in L
            s = A[i, k] / A[k, k]

            # Update the current row, starting from column k
            for j in range(k, n):
                A[i, j] = A[i, j] - s * A[k, j]

            # Store the elimination multiplier in L
            L[i, k] = s

    # The diagonal entries of L should be 1
    L += np.eye(m)

    # At this point, A has become the upper triangular matrix U
    return L, A

"""这个函数对下三角矩阵形式的 Ax = b 做前代，求出中间变量向量 x。思路是从第一行开始，
# This function performs forward substitution on a lower triangular syste
def forward_substitution(A, b):

    # Number of equations
    n = np.size(b)

    # Initialize the solution vector
    x = np.zeros(n)

    # Perform forward substitution from the first row downward
    for k in range(n):

        # Stores the sum of known terms
        s = 0.0
```

```
        # Sum the known terms A[k, j] * x[j] for j < k
        for j in range(k):
            s = s + A[k, j] * x[j]

        # Solve for x[k] using the forward substitution formula
        x[k] = (b[k] - s) / A[k, k]

    return x

"""这个函数对上三角矩阵形式的 Ax = b 做回代，求出解向量 x。思路是从最后一行开始，逐步
# This function performs back substitution on an upper triangular system
def backward_substitution(A, b):

    # Number of equations
    n = np.size(b)

    # Initialize the solution vector
    x = np.zeros(n)

    # Perform back substitution from the last row upward
    for k in range(n - 1, -1, -1):

        # Stores the sum of known terms
        s = 0.0

        # Sum the known terms A[k, j] * x[j] for j > k
        for j in range(k + 1, n):
            s = s + A[k, j] * x[j]

        # Solve for x[k] using the back substitution formula
        x[k] = (b[k] - s) / A[k, k]

    return x

"""完整的 LU 求解函数。它先对 A 做 LU decomposition，再先解 Ly = b（forward sub
# This is a complete LU solve function. It first performs LU decompositio
def LU_solve(A, b):

    # First decompose A into L and U
    L, U = LU_decomposition(A)

    # Print L and U for inspection
    print('L =', L)
    print('U =', U)

    # Step 1: solve Ly = b
    y = forward_substitution(L, b)

    # Step 2: solve Ux = y
    x = backward_substitution(U, y)

    return x

# 用我们自己写的 LU_solve 函数求解
# Solve the system using our own LU_solve function
x = LU_solve(A, b)

# 输出解向量
# Print the solution vector
print('x =', x)
```

```
# 用 NumPy 内置求解器进行验算
# np.allclose(...) 用来检查两个结果是否在数值误差允许范围内一致
# Verify the result using NumPy's built-in solver
# np.allclose(...) checks whether the two results agree within numerical
print(np.allclose(np.linalg.solve(A, b), LU_solve(A, b)))
```

4.3 Gaussin-Seidal method

In [ ]:

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.linalg as sl
import scipy.optimize as sop

# 此处记得更换数据，A 是方程左边各变量的系数矩阵，b 是右边的常数向量
# Gauss-Seidel Method
# A is the coefficient matrix on the left-hand side, and b is the constan
A = np.array([[2.,  3.,  1.,  5.],
              [8., 16.,  7., 22.],
              [2., 15., 13., 19.],
              [-4., 2., 10., 11.]])
b = np.array([4., 24., 30., 13.])

# 参数设置，tol 是收敛容差，当 residual 小于这个值时就停止迭代，it_max 是最大迭代次
# parameter settings, tol is the convergence tolerance; iteration stops o
tol = 1.e-6
it_max = 10000

"""这个函数使用高斯-赛德尔迭代法来求解线性方程组 Ax = b。它会在每一步更新解向量 x，并
# This function solves the linear system Ax = b using the Gauss-Seidel it
def gauss_seidel(A, b, it_max, tol):

    # m is the number of equations (rows), n is the number of unknowns (c
    m, n = A.shape

    # Initialize the solution vector x with zeros as the initial guess
    x = np.zeros(A.shape[0])

    # Store the residual after each iteration
    residuals = []

    # Start the iteration, with at most maxit iterations
    for k in range(it_max):

        # Update each unknown x[i] one by one
        for i in range(m):

            # Gauss-Seidel formula:
            # x[i] is computed using the newly updated values x[:i]
            # and the not-yet-updated old values x[i+1:].
            x[i] = (1. / A[i, i]) * (
                b[i]
                - np.dot(A[i, :i], x[:i])        # already updated part
                - np.dot(A[i, i+1:], x[i+1:])    # not-yet-updated part
            )

        # Compute the current residual ||Ax - b||
        residual = sl.norm(A @ x - b)
```

```
        # Append the current residual to the list
        residuals.append(residual)

        # Stop if the residual is already small enough
        if residual < tol:
            break

    # Return the final solution vector and the residual history
    return x, residuals

# 初始猜测向量，这里全设成 0，执行高斯-赛德尔迭代并且输出计算得到的解
# Initial guess for the solution, here chosen as a zero vector
# Run the Gauss-Seidel iteration
# Print the computed solution
x = np.zeros(A.shape[0])
x_gs, res_gs = gauss_seidel(A, b, it_max, tol)
print(x_gs)

# 画出 residual 随迭代次数变化的图，semilogy 表示 y 轴取对数刻度，这样更容易看出 re
# Plot the residual against iteration number,  semilogy means the y-axis
fig = plt.figure(figsize=(6, 4))
ax1 = plt.subplot(111)
ax1.semilogy(res_gs, 'b.-', label='Gauss-Seidel')
ax1.set_xlabel('Iteration')
ax1.set_ylabel('Residual')
ax1.set_title('Convergence plot')
ax1.legend(loc='best')
plt.show()

# 用逆矩阵方法做一个简单验算
# gauss_seidel(A, b) 返回两个量：
# 1. 解向量 x
# 2. residual 列表
# 所以这里只取 [0]，也就是解向量部分。
# Perform a simple verification using the inverse-matrix method
# gauss_seidel(A, b) returns two objects:
# 1. the solution vector x
# 2. the residual history
# So here [0] is used to extract only the solution vector.
print(np.allclose(sl.inv(A) @ b, gauss_seidel(A, b, it_max, tol)[0]))
```